I.2. Pipeline - rezolvarea problemelor

Avansarea (1)

```
add dword ptr [eax],5
sub ecx,[eax]
```

- rezultatul adunării - calculat de ALU
- durează până când este scris la destinaţie
- instrucţiunea de scădere poate prelua rezultatul adunării direct de la ALU

Avansarea (2)

Avantaj

- reduce timpii de aşteptare

Dezavantaje

- necesită circuite suplimentare complexe
- trebuie considerate relaţiile între toate instrucţiunile aflate în execuţie (în pipeline)

Unităţi superscalare

- ideea de bază - mai multe ALU
- se pot efectua mai multe calcule în paralel
- folosită împreună cu tehnica pipeline
- MAR şi MDR nu pot fi multiplicate
- cât de mult se pot multiplica ALU?
  - depinde de structura şi eficienţa pipeline

Unităţi superpipeline

- mai multe pipeline în acelaşi procesor
  - de obicei 2
- 2 (sau mai multe) instrucţiuni executate complet în paralel
- restricţii
  - accesele la memorie şi periferice - secvenţial
  - unele instrucţiuni pot fi executate de un singur pipeline

Predicţia salturilor (1)

- rezolvarea dependenţelor de control
- ideea de bază - a "prezice" dacă un salt se execută sau nu
  - nu se aşteaptă terminarea instrucţiunii de salt
- predicţie corectă - fără blocaje în pipeline
- predicţie eronată - se execută instrucţiuni care nu trebuiau executate
  - efectul acestora trebuie anulat

Predicţia salturilor (2)

- spor de performanţă - cât mai multe predicţii corecte (nu neapărat 100%)
- o instrucţiune executată eronat produce efecte doar când rezultatul este scris la destinaţie
- rezultatele instrucţiunilor - memorate intern de procesor până când se verifică dacă predicţia a fost corectă

Scheme de predicţie

Tipuri de scheme

- statice
  - întotdeauna aceeaşi decizie
- dinamice
  - se adaptează în funcţie de comportarea programului

Scheme statice de predicţie

- saltul nu se execută niciodată
  - rata predicţiilor corecte ≈ 40%
- saltul se execută întotdeauna
  - rata predicţiilor corecte ≈ 60%
- salturile înapoi se execută întotdeauna, cele înainte niciodată
  - combină variantele anterioare
  - rată superioară a predicţiilor

Scheme dinamice de predicţie (1)

- procesorul reţine într-un tabel comportarea la salturile anterioare
  - salt executat/neexecutat
- un singur element pentru mai multe instrucţiuni de salt
  - tabel mai mic → economie de spaţiu

Scheme dinamice de predicţie (2)

Tipuri de predictori

- locali
  - reţin informaţii despre salturile individuale
- globali
  - iau în considerare corelaţiile dintre instrucţiunile de salt din acelaşi program
- micşti

Utilizarea cache-ului în predicţie

Cache-ul de instrucţiuni

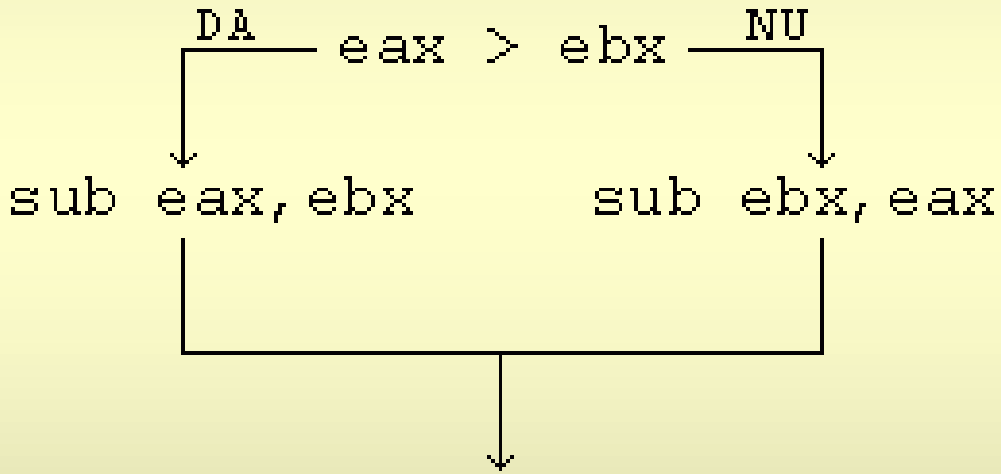
- reţine vechea comportare a unui salt
  - condiţie
  - adresă destinaţie
- *trace cache*
  - memorează instrucţiunile în ordinea în care sunt executate
  - nu în ordinea fizică

Execuţia speculativă

- înrudită cu predicţia salturilor
- se execută toate variantele posibile
  - înainte de a şti care este cea corectă
- când se cunoaşte varianta corectă, rezultatele sale sunt validate
- se poate utiliza şi în circuite simple

Cum funcţionează? (1)

- instrucţiuni de salt condiţionat



- ambele variante se execută în paralel
- cum se modifică regiştrii `eax` şi `ebx`?

Cum funcţionează? (2)

- nici una din variante nu îi modifică
- rezultatele scăderilor sunt depuse în regiştri temporari
- când se cunoaşte relaţia între `eax` şi `ebx`
  - se determină varianta corectă de execuţie
  - se actualizează valorile `eax` şi `ebx` conform rezultatelor obţinute în varianta corectă

Execuţie speculativă vs. predicţie

- nu apar predicţii eronate
  - rata de succes - 100%
- necesită mulţi regiştri pentru rezultatele temporare
- gestiunea acestora - complicată
- fiecare variantă de execuţie poate conţine alte salturi etc.
  - variantele se multiplică exponenţial

Predicaţia (1)

- folosită în arhitectura Intel IA-64
  - şi în alte unităţi de procesare
- asemănătoare cu execuţia speculativă
- procesorul conţine regiştri de predicate
  - predicat - condiţie booleană (bit)
- fiecare instrucţiune obişnuită are asociat un asemenea predicat

Predicaţia (2)

- o instrucţiune produce efecte dacă şi numai dacă predicatul asociat este *true*
  - altfel rezultatul său nu este scris la destinaţie
- instrucţiunile de test pot modifica valorile predicatelor
- se pot implementa astfel ramificaţii în program

Exemplu

- pseudocod

```
if(R1==0){
  R2=5;
  R3=8;
  }
else
  R2=21;
```

Exemplu (continuare)

- limbaj de asamblare "clasic"

```
    cmp R1,0
    jne E1
    mov R2,5
    mov R3,8
    jmp E2
E1: mov R2,21
E2:
```

Exemplu (continuare)

- limbaj de asamblare cu predicate

```
    cmp R1,0,P1
<P1>mov R2,5
<P1>mov R3,8
<P2>mov R2,21
```

- predicatele P1 şi P2 lucrează în pereche
  - P2 este întotdeauna inversul lui P1
  - prima instrucţiune îi modifică pe amândoi

Execuţia out-of-order

- instrucţiunile nu se mai termină obligatoriu în ordinea în care şi-au început execuţia
- scop - eliminarea unor blocaje în pipeline
- posibilă atunci când între instrucţiuni nu există dependenţe

Exemplu

```
in al,278
add bl,al
mov edx,[ebp+8]
```

- prima instrucţiune - foarte lentă
- a doua instrucţiune trebuie să aştepte terminarea primeia
- a treia instrucţiune nu depinde de cele dinaintea sa - se poate termina înaintea lor

Redenumirea regiştrilor

- dependenţe de date
  - apar când două instrucţiuni folosesc aceeaşi resursă (variabilă/registru)
  - numai când cel puţin una din instrucţiuni modifică resursa respectivă
  - dacă resursele sunt regiştri, unele dependenţe se pot rezolva prin redenumire

Tipuri de dependenţe de date

- RAW (*read after write*)
  - prima instrucţiune modifică resursa, a doua o citeşte
- WAR (*write after read*)
  - invers
- WAW (*write after write*)
  - ambele instrucţiuni modifică resursa

Dependenţe RAW

- dependenţe "adevărate"
- nu pot fi eliminate

```
mov eax,5
sub ebx,eax
```

- valoarea scrisă în `eax` de prima instrucţiune este necesară instrucţiunii următoare

Dependenţe WAR

- numite şi antidependenţe

```
add esi,eax
mov eax,16
sub ebx,eax
```

- prima instrucţiune trebuie executată înaintea celei de-a doua (nu se poate în paralel)

Dependenţe WAR - rezolvare

```
add esi,eax          add esi,eax
mov eax,16   ──→     mov reg_tmp,16
sub ebx,eax          sub ebx,reg_tmp
```

Dependenţe WAW

- dependenţe de ieşire

```
div ecx
sub ebx,edx
mov eax,5
add ebp,eax
```

Dependenţe WAW - rezolvare

```
div ecx                div ecx,3
sub ebx,edx            sub ebx,edx
mov eax,5      →       mov reg_tmp,5
add ebp,eax            add ebp,reg_tmp
```

Utilitate

- ajută la creşterea performanţei?
- creşte potenţialul de paralelizare
- mai eficientă în combinaţie cu alte tehnici
  - structura superpipeline
  - execuţia out-of-order

Eficienţa abordării

- mai mulţi regiştri
  - folosiţi intern de procesor
  - nu sunt accesibili programatorului
- de ce?
  - redenumirea se face automat
  - programatorul poate greşi (exploatare ineficientă a resurselor)
  - creşterea performanţei programelor vechi